Course TA F0 Project Report

CST3492 Java Enterprise Application Development
Assignment A | Group B | Academic Year 2026 2027 Semester 1

Release 0.1.0 | 29 October 2026

1 Team particulars

Full name	Matriculation number
_____	_____
_____	_____
_____	_____
_____	_____

Contact number: _____
Email address: _____

Additional team members or particulars: _____
_____

Project scope

Course TA implements the F0 teaching-assistant functions inside IntelliJ IDEA. It retrieves passages from local course materials, sends relevant evidence and the question to a configurable LLM service, and associates course answers with document and page references. Students can also attach selected editor code to a question.

This report describes the F0 release. Group B also requires F1 before its final Assignment A submission. The F1 smart actions are outside this release.

The report follows the assignment order: team particulars, implemented functions, user manual, architecture and technical design, then other technical information. Member and contact fields are intentionally left for the team to complete.

2 Implemented functions

F0 requirement	Implemented behavior
Course material preprocessing	Recursively imports text-based PDF, UTF-8 TXT and Markdown. PDFBox extracts every page separately. Long passages are split with overlap.
Searchable knowledge base	A local BM25 inverted index ranks passages; a versioned JSON snapshot is stored per IDEA project.
Retrieval augmented answers	The question and up to four relevant excerpts are passed to a configurable Chat Completions service.
Source citation	Prompt IDs such as [S1] map to relative document names and physical PDF page numbers or text line ranges. Missing or invalid IDs cause the answer to be withheld.
General knowledge fallback	When no evidence is retrieved, the displayed answer always includes the required general-knowledge notice.
Embedded interface	A Course TA tool window provides conversation text, settings, import, question entry, retrieval preview and cancellation.
Editor context awareness	An editor right-click action captures selected text, filename and the exact inclusive line range. The user reviews the attachment before sending.

Supporting behavior

API keys use IDEA PasswordSafe. Long operations run on a background executor; UI updates return to the event dispatch thread. Rebuilds replace the stored index after extraction succeeds. Failed imports retain the previous usable index. Import warnings and HTTP failures are shown in the conversation.

Retrieval preview operates locally and labels itself as a preview rather than an LLM response. Chat text is kept in memory. Each question is an independent request; previous turns are not included automatically.

Requirement boundary

This release does not implement scenario-based JUnit generation, staged-diff commit messages or the creative F1 action. It also does not implement the Group C diff-approval workflow.

3 User manual

3 1 Installation and configuration

Requirements. Use IntelliJ IDEA 2026.1.x and JDK 21 or later for building. The distributed plugin targets Java 21 bytecode and IDE build 261.*. The project includes a Gradle wrapper and a Windows helper that can reuse an installed IDEA SDK.

Build. Open the source folder in PowerShell and run `.\tools\build.ps1 -Task Build`. The plugin ZIP is written to `build/distributions/course-ta-0.1.0.zip`. First-time dependency downloads require network access.

Install. In IDEA, open Settings, Plugins, the gear menu, and Install Plugin from Disk. Select the plugin ZIP and restart if prompted. Open a local project, then View, Tool Windows, Course TA.

Import. Click Import / Rebuild and select the supplied course-materials folder. It contains five lecture PDFs. Wait for the file, page and chunk counts. Rebuild after changing the corpus. Avoid mixing assignment instructions or exam documents into the lecture folder.

Configure the model. Click Settings, provide the full Chat Completions endpoint, a valid model name and the API key. The initial endpoint is https://api.deepseek.com/chat/completions. Use the model supported by the selected provider. A blank key field keeps the stored key; Remove stored key deletes it.

Ask a question. Enter a question and click Ask TA. The configured provider receives the question, attached code and up to four course excerpts. Preview retrieval displays local evidence without contacting the provider.

3 2 Answers and references

Course answers use source markers such as [S1], followed by a source list. A label such as 2.pdf | p. 22 means the twenty-second physical page of that PDF. Printed slide labels can differ. Text files use inclusive line ranges. Use the original PDF viewer to inspect the referenced page.

If the answer is generated without course evidence, the interface displays: *Response is based on general knowledge; no specific course material is referenced.* Invalid or omitted source IDs lead to a visible validation message and a retry request.

3 3 Recovery

Cancel interrupts the current task. Authentication failures require checking credentials; quota or rate-limit errors require checking provider limits. Corrupt PDFs and empty pages produce import warnings. An image-only PDF needs OCR before import. Clear chat removes the visible history, and Clear code removes the current attachment.

3 4 Retrieval interface sample

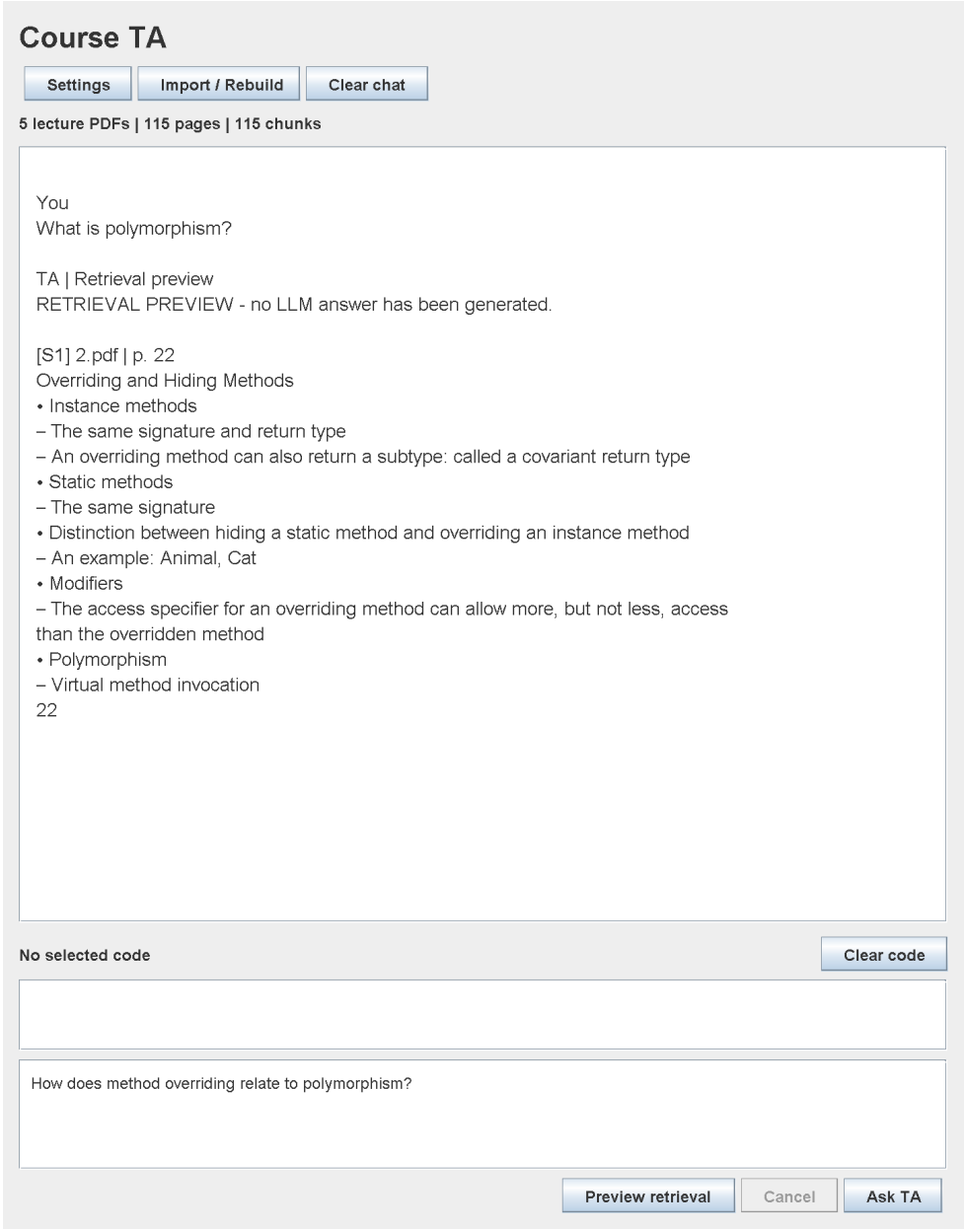


Figure 1. The production Swing tool-window component rendered offscreen with the actual course index. This is an interface sample, not an IDE screenshot or a live LLM response.

The upper toolbar controls configuration and import. Retrieved passages retain source labels. The lower composer provides a question field, cancellation and separate preview and LLM actions.

3 5 Questions about selected code

Select a Java method or statement, right-click Ask Course TA About Selected Code, and enter a question. The tool window displays the selected text and line range. Review them, then click Ask TA. The action does not alter the source file.

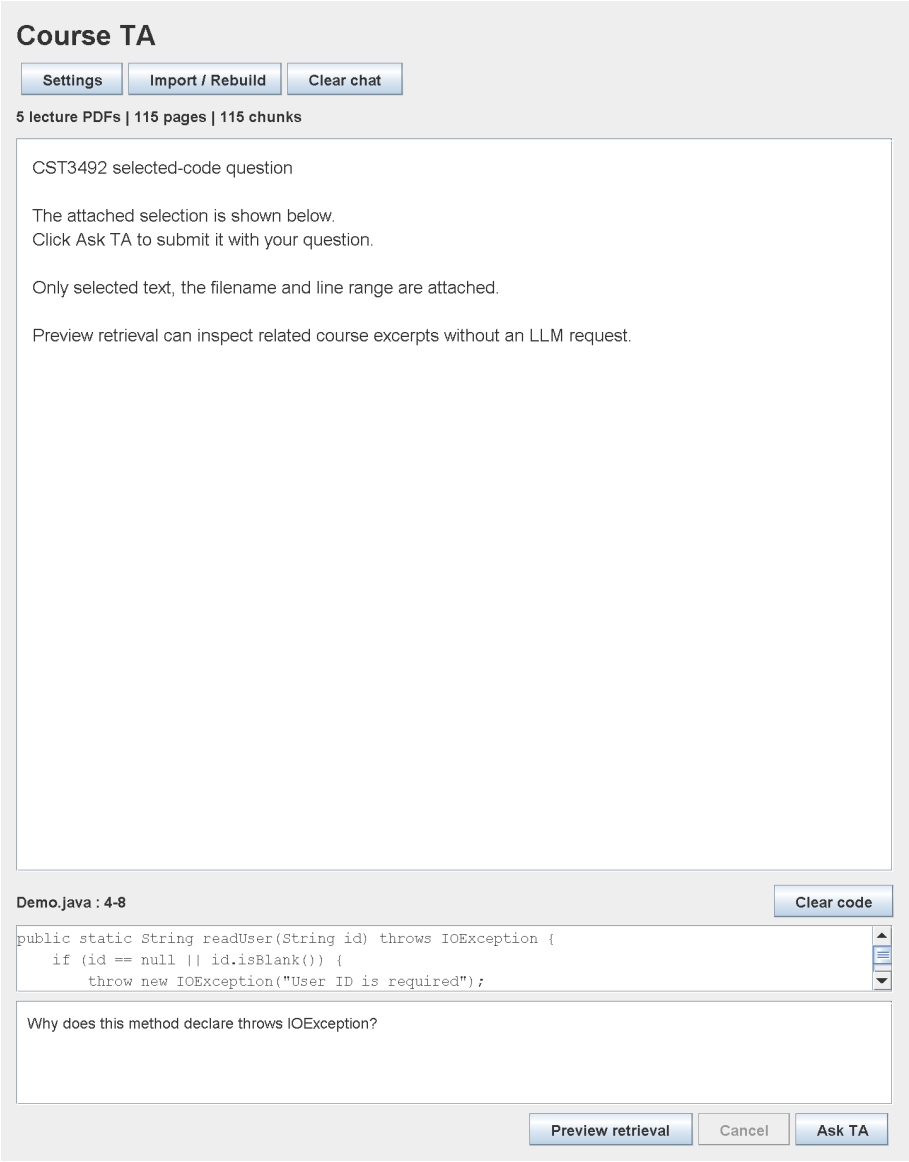


Figure 2. Production composer with a selected-code attachment. The sample uses Demo.java. The context action also works while the IDE is indexing.

Selections are limited to 12,000 characters; questions are limited to 6,000. Only the selected code is attached. The remaining file and unrelated project files are not read or sent.

4 Architecture and workflow

Layer	Components and responsibility
IDE integration	plugin.xml registers the tool window and editor action. TaToolWindowFactory constructs the panel. AskSelectedCodeAction captures editor context.
Presentation and control	ChatView owns Swing widgets. TaPanel routes events, runs background work and updates status.
Project services	TaSettings stores ordinary settings. TaProjectService manages the per-project index and PasswordSafe access.
RAG core	DocumentIngestor, Chunk, Bm25Index and IndexStore extract, represent, retrieve and persist course evidence.
LLM and attribution	RagAssistant assembles the prompt and checks references. LlmClient performs the HTTP request and parses the response.

4 1 Index construction

Folder selection → supported-file discovery → page or line extraction → overlapping chunks → BM25 postings → versioned JSON snapshot. The snapshot is written through a temporary file and replaced atomically where the filesystem supports it. Individual unreadable files produce warnings; a rebuild with no usable text leaves the previous index intact.

4 2 Question answering

Question and optional selection → lexical retrieval → relevance gate → up to four excerpts → system and user messages → LLM response → citation validation → displayed answer and source list. If the question returns no evidence and code is attached, the selection is used as a second retrieval query. If both searches return nothing, the general-knowledge branch is used.

4 3 State and storage

Structure	Fields and lifetime
Chunk	document, location, text; immutable record.
Posting	Chunk integer ID and term frequency; in-memory lists grouped by token.
Index snapshot	version = 1 and chunks; .idea/course-ta/index.json in the opened project. Postings are rebuilt on load.
Configuration	Course-folder path, endpoint and model in IDEA project state. API key in PasswordSafe, associated with the endpoint.
Conversation and code	In-memory UI state. Each request captures the question and selection before background execution.

4 4 Chunking and retrieval design

PDFBox 3 extracts text one physical PDF page at a time, with positional sorting enabled. Each chunk is at most 1,400 Java characters, with a target overlap of 200 characters. A newline in the latter half of the window is preferred as the boundary. A chunk never crosses a PDF page, making the citation scope deterministic.

UTF-8 TXT and Markdown are grouped by original lines. Adjacent blocks share up to two lines; unusually long blocks are subdivided with the same character budget and retain their enclosing line scope. File paths are relative to the imported folder, preserving distinctions between files with identical names in different subfolders.

Tokenization and BM25

Tokenization splits camel case, lowercases text, extracts Unicode letter and number sequences, removes common stop words and applies a simple plural suffix rule. A small dictionary adds English equivalents for common Chinese Java topics, including exceptions, inheritance and polymorphism. This is a limited lexical aid rather than general machine translation.

For each distinct query term, the index visits its posting list. BM25 uses k1 = 1.2 and b = 0.75. A term contribution increases with frequency, is normalized by chunk length, and is weighted more strongly when the term occurs in fewer chunks. The smoothed IDF is ln(1 + (N - df + 0.5) / (df + 0.5)).

Candidates must have a total BM25 score of at least 1.0 and match at least 20% of distinct query terms. Results are sorted by score, with chunk order as a stable tie-breaker, and limited to four. These are transparent heuristics; they can reject a useful paraphrase or accept an incidental lexical match.

Efficiency and rationale

An inverted index limits query scoring to chunks containing query terms. Index construction is linear in extracted token occurrences; storage is proportional to chunk text and postings. A course-sized corpus fits in memory and needs no separate database, embedding service or embedding API key. Reloading a JSON snapshot avoids repeating PDF extraction, but rebuilds postings.

At most 5,600 excerpt characters, a 6,000-character question and 12,000 characters of selected code enter a prompt, plus metadata and instructions. Character limits are not token limits. Model-specific context windows still need to be respected.

The design favors a reproducible F0 implementation with explicit provenance. An embedding or hybrid retriever, a reranker and incremental hashing are possible extensions when the corpus or query variety grows.

4 5 LLM integration and attribution

The client sends a non-streaming JSON Chat Completions request with a model, temperature 0.2, and system/user messages. It uses a 15-second connection timeout and a 90-second request timeout. HTTPS is required for remote endpoints; HTTP is accepted for localhost. Responses are read from choices[0].message.content.

Prompt contract

The system message defines the course TA role, asks for the question language, requires source IDs for course-based claims, prohibits invented references, and separates course evidence from general Java knowledge. Retrieved excerpts and code are labeled as untrusted data rather than instructions. The user message contains the original question, selection metadata and individually labeled excerpts.

A retrieved block has the form `[S1] 2.pdf | p. 22`, followed by its text. The model returns inline markers such as `[S1]`. Source metadata is controlled by the application; it is not generated by the model.

Citation checks

The application checks every source marker against the retrieved set. An out-of-range or unparseable ID withholds the answer. If evidence is present but the model provides no source marker, the answer is withheld unless it explicitly declares the general-knowledge fallback. If there are no retrieved passages, the application adds the exact fallback notice regardless of whether the model remembered it.

These checks establish that an ID resolves to supplied evidence. They do not prove semantic entailment, verify every sentence or detect all misleading claims. The student should inspect the cited passages. Retrieved sources that were supplied but not cited remain labeled as retrieved sources.

Responsiveness and handling errors

PDF extraction, index I/O, PasswordSafe access and HTTP calls run off the event dispatch thread. A task generation number prevents cancelled or superseded jobs from appending stale results. Project disposal cancels the active future. Rebuild cancellation is checked between files/pages and before saving.

HTTP 401/403 returns a credential message; 429 returns a rate/quota message; other unsuccessful status codes return a generic provider failure. Provider error bodies, prompts and credentials are not logged. Missing response content produces a compatibility error. The plugin does not execute model-generated code.

5 Validation and technical information

5 1 Automated verification

The core JUnit suite ran 18 tests with 0 failures, 0 errors and 0 skipped tests. The tests cover page-preserving PDF parsing, chunk limits and overlap, UTF-8 line scopes, corrupt files, cancellation, ranking, no-hit queries, Chinese topic aliases, snapshot persistence, prompt bounds, citation checks, endpoint validation and local HTTP integration.

HTTP tests use an in-process server with a deterministic response. They verify request construction and response handling, not the quality of a commercial model. No real provider credential was supplied for this release verification.

5 2 Full course corpus audit

The audit read all 5 supplied lecture PDFs, covering 115 physical pages and producing 115 chunks. It recorded 0 warnings. A SHA-256 digest is recorded for each input. Import plus indexing took 1,648 ms in this single local run; this is an observation, not a benchmark.

Question	Top retrieved source
What is polymorphism?	2.pdf | p. 22
How do checked exceptions work?	3.pdf | p. 6
What is retrieval augmented generation?	0.pdf | p. 3
Explain constructors	2.pdf | p. 23
Chinese query about polymorphism	2.pdf | p. 22
astronomy galaxy telescope	No relevant material

Audit evidence is in course-audit.json; test evidence is in the JUnit XML report. The index and original PDF hashes make the retrieval examples reproducible.

5 3 Release checks and limits

The release includes a compiled plugin ZIP, source code and the production UI component samples. The build targets the installed IDEA 2026.1.1 SDK. Interactive IDE behavior and a real LLM response must still be exercised in the team demonstration; offscreen UI rendering is not an IDE end-to-end test.

Remaining limits include no OCR, lexical rather than semantic retrieval, no automatic corpus watching, independent rather than conversational requests, and no semantic verification of generated claims. Course materials and selected code are transmitted only when Ask TA is used.

5 4 Demonstration and next milestone

Suggested F0 demonstration

1. Open the plugin, import the five lecture PDFs and show the resulting counts.
2. Preview a polymorphism query and open the cited lecture page.
3. Configure the team provider and ask a course question; inspect inline citations.
4. Select the sample method, use the editor action and ask about checked exceptions.
5. Ask an unrelated question and show the general-knowledge notice.
6. Explain the index, prompt construction and citation gate; identify the limitations.

Team collaboration record

Implementation responsibilities: _____
Review and testing responsibilities: _____
Documentation and presentation responsibilities: _____
Collaboration method and evidence: _____

Complete these fields from actual team contributions. The report makes no claim about member work allocation or a presentation that has not occurred.

Group B F1 milestone

The next release must add the three Group B smart actions: JUnit tests generated from a user-entered scenario and selected code; commit messages based on all staged diffs; and one creative smart action. The core HTTP client and IDE action layer can be reused. Update the feature list, manual, validation evidence and presentation when these are implemented.

References

Assignment A specification. CST3492, AY 2026-2027 Semester 1, pp. 1-4.

Course lectures. Dr. Fan Hongfei: 0.pdf, 1.pdf, 2.pdf, 3.pdf and Lec-4-AI-Assisted-Java-Development.pdf. Original course files supplied for this project.

IntelliJ Platform Gradle Plugin. https://plugins.jetbrains.com/docs/intellij/tools-intellij-platform-gradle-plugin.html

IntelliJ tool windows. https://plugins.jetbrains.com/docs/intellij/tool-windows.html

PasswordSafe. https://plugins.jetbrains.com/docs/intellij/persisting-sensitive-data.html

Apache PDFBox 3. https://pdfbox.apache.org/3.0/migration.html

DeepSeek Chat Completions. https://api-docs.deepseek.com/api/create-chat-completion/